# SOFTWARE CONFIGURATION

## Python Installation:

### 1. Download Python 3.7.6

- Go to the official Python website:

  https://www.python.org/downloads/release/python-376/

- Download the appropriate installer for your operating system

  (Windows/macOS/Linux).

### 2. Install Python for windows:

- Run the downloaded **.exe** file.
- Check the box "**Add Python to PATH**" before proceeding.
- Click **Install Now** and follow the installation wizard.

### 3. Verify Installation:

- Use the following command in the terminal: "**python --version**"
- It should display Python 3.7.6

## Environment Files:

The environment setup consists of these dependencies and configurations:

- **A dependency file (set1.txt)** for installing required Python packages.
- **A script (nltkdownload.py)** to download necessary NLP datasets.
- **A Django management file (manage.py)** for running the web framework.

### 1. Dependency file Installation (set1.txt):

**"** pip install -r .\set1.txt **"**

- **pip**: Python package manager used for installing libraries.
- **install**: Command to install packages.
- **-r**: Reads dependencies from a file.
- **.\set1.txt**: Specifies the file containing the list of required packages.

2. **NLP Dataset Downloader file (nltkdownload.py):**

**“ python -m nltk.downloader all ”**

- **python**: Calls the Python interpreter.
- **-m**: Runs a module as a script.
- **nltk.downloader**: The module for downloading NLTK datasets.
- **all**: Downloads all available corpora and resources.

## Django Environment Setup:

To set up a Django project, a series of commands are executed to collectively establish a Django development environment. Below is a explanation of each command that should be executed in sequence to form a proper Django directory structure:

1. **python -m venv environment**
   - This command creates a virtual environment named **"environment"** using Python's built-in venv module.
2. **. \environment\Scripts\activate**
   - This activates the virtual environment **"environment"**, allowing Python and pip to use the environment's dependencies instead of system-wide packages.
   - On macOS/Linux, the command would be source environment/bin/activate.
3. **pip install Django**
   - Installs Django inside the virtual environment to ensure the project uses a consistent version of the framework.
4. **django-admin --version**
   - Checks if Django was installed correctly by displaying the installed version.
5. **django-admin startproject Deep**
   - Creates a new Django project named **"Deep"**.
   - This command generates essential project files and initial configurations.

6. **cd Deep**
    - Navigates into the newly created project directory **"Deep"**, allowing further configurations and application setup.
7. **django-admin startapp Deepapp**
    - Creates an application named **"Deepapp"** inside the project.
    - In Django, an application represents a functional module within the project (e.g., user authentication, blog, etc.).
8. **python manage.py runserver**
    - Starts the Django development server, allowing developers to access the project in a web browser at http://127.0.0.1:8000/.